

Programmazione Funzionale

Appunti per l'orale di Programmazione Funzionale

Anno Accademico: 2024/25

Mattia Cosimi

*Dipartimento di Ingegneria Civile, Informatica e delle Tecnologie Aeronautiche
Università degli Studi "Roma Tre"*

Indice

1	Nozioni iniziali	1
1.1	Forma Currificata	1
1.2	Tipi	1
1.3	Ricorsione	2
2	Liste	2
2.1	Definizione di nuovi tipi	2
3	Alberi	2
3.1	Definizione degli alberi binari per induzione strutturale	3
3.2	Definizione degli alberi n-ari per induzione strutturale	3

1 Nozioni iniziali

Durante la **modalità interattiva** il compilatore legge (read) un'espressione o una dichiarazione, calcola il valore dell'espressione (eval) e ne deduce il tipo, infine stampa il valore dell'espressione (print). Ocaml è un linguaggio a **tipizzazione statica** ovvero ogni espressione ha un tipo che può essere determinato a tempo di compilazione. Le **espressioni** hanno sempre un valore e un tipo. Il calcolo procede **valutando espressioni** che significa semplificarla fino ad ottenere un'espressione non più semplificabile ovvero un valore. Ocaml ha un **ambiente di valutazione** che è una collezione di legami variabile-valore. L'ambiente viene gestito come una **pila**, i nuovi legami vengono aggiunti in alto e si cerca il valore delle variabili a partire dall'alto. Quando viene dichiarata una funzione che utilizza delle variabili, il valore delle variabili verrà cercato nell'**ambiente di dichiarazione** della funzione ovvero nell'ambiente in cui è stata definita (tutto ciò che la precede nella pila). Quando una funzione viene applicata a un argomento, il suo argomento viene valutato nell'ambiente e viene creato un nuovo legame **provvisorio** del parametro formale con il valore dell'argomento. Successivamente il legame provvisorio viene cancellato. In Ocaml è presente il polimorfismo in particolari ci sono le **variabili di tipo** ('a). Per quanto riguarda l'uso delle parentesi: nei tipi (**espressioni di tipo**) si associa a destra

$$int \rightarrow int \rightarrow int = int \rightarrow (int \rightarrow int)$$

Nelle **espressioni** si associa a sinistra

$$(times\ 3)\ 5 = times\ 3\ 5$$

1.1 Forma Currificata

Per avere una **forma currificata** basta far consumare alla funzione un'argomento alla volta (*function* $\rightarrow$). In questo modo le funzioni possono essere applicate parzialmente:

$$sum\ (times\ 5)\ 1\ 10 = \sum_{k=1}^{10} (5 * k)$$

In generale per currificare una funzione deve esser possibile omettere le parentesi in modo tale da consumare gli argomenti uno alla volta.

1.2 Tipi

Nel caso dei tipi booleani, Ocaml ha una valutazione "pigra", se la prima condizione dell (and) è vera allora valuta la seconda, se la prima condizione dell (or) è falsa allora valuta la seconda. Gli interi lavorano con le operazioni per gli interi (+ - */) i float con quelle per i float (+. -. *. /.). In Ocaml, non c'è la conversione automatica dei tipi numerici. Uguaglianza, disuguaglianza e operatori di confronto sono definiti su qualsiasi tipo, eccetto le funzioni e i tipi contenenti funzioni. Nell'if i rami devono avere lo stesso tipo. Ocaml applica la regola di calcolo **Call by value** la quale calcola il valore dell'argomento prima di applicare una funzione. Ogni tipo di dati è caratterizzato da un

insieme di **costruttori** (costanti + operazioni che "costruiscono" valori di quel tipo) (esempio `(,)` parentesi e virgola costruiscono una coppia) e un insieme di **selettori** (`fst`, `snd`) (operazioni che "selezionano" componenti da un valore del tipo).

1.3 Ricorsione

La ricorsione è una tecnica per risolvere problemi complessi riducendoli a problemi più semplici dello stesso tipo. Per risolvere un problema ricorsivamente: **1.** identificare i casi base, **2.** identificare i problemi più semplici dello stesso tipo la cui soluzione può aiutare a risolvere il problema complesso, **3.** assumendo di saper risolvere i problemi più semplici, determinare come operare sulla soluzione dei problemi più semplici per ottenere la soluzione del problema complesso. Quando definiamo un `let x = E in F` ecc `x` è una **variabile locale** e come tale quando tutta l'espressione è stata valutata `x` non ha più un valore. Quando abbiamo una sequenza di espressioni il compilatore le valuta tutte ma i valori vengono ignorati tranne quello dell'ultima espressione. Le funzioni ricorsive possono essere **tail recursive** o **iterative**. In un processo **ricorsivo** i calcoli vengono eseguiti al ritorno della ricorsione, usa spazio proporzionale al numero di chiamate ricorsive. In un processo **iterativo** i calcoli vengono eseguiti tutti prima della chiamata ricorsiva e il risultato parziale viene conservato in un **accumulatore**.

2 Liste

I **costruttori** del tipo `'a list` sono: `[]` e `::` mentre i **selettori** sono **`List.hd`** (restituisce la testa della lista) e **`List.tl`** (restituisce la lista senza la testa). Le liste vengono utilizzate per rappresentare dati come i dizionari che sono una collezione di elementi (chiave, valore). Il **Backtracking** consiste nel costruire la sequenza aggiungendo un elemento alla volta e controllare passo passo se la sequenza parziale ha possibilità di successo. Ad ogni stadio `i`: se $x_1, \dots, x_i$ ha possibilità di successo, si sceglie un elemento x_{i+1} , tra le diverse alternative; se, con tale scelta, si arriva a una soluzione sol quella è la soluzione. Altrimenti si sceglie un diverso x_{i+1} . Se non si trova una soluzione dopo aver esaminato tutte le possibilità: fallimento.

2.1 Definizione di nuovi tipi

Per definire un nuovo tipo occorre: un nome per il tipo e come costruire i valori del tipo, cioè quali sono i **costruttori** del tipo. Ovviamente i tipi possono essere definiti anche ricorsivamente. Un'altra cosa particolare è il tipo `type 'a option = None | Some of 'a` utile quando non si vuole la propagazione degli errori.

3 Alberi

Un'albero è un insieme di oggetti, chiamati **nodi**, su cui è definita una relazione binaria $G(n, m)$ che leggiamo con `n` è genitore di `m`. Esiste un unico nodo, chiamato **radice**, che non ha genitori. Ogni nodo diverso dalla radice ha un unico genitore. Per ogni nodo `n` diverso dalla radice esiste

un **cammino** dalla radice a n (l'albero è connesso): esistono nodi $n_1, \dots, n_k$ ($k \geq 1$) tali che n_1 è la radice dell'albero, $n_k = n$ e, per ogni $i = 1, \dots, k-1$, n_i è genitore di n_{i+1} . Se n è il genitore di m , allora m è un **figlio** di n . Un **albero binario** è un albero in cui ogni nodo ha al massimo due figli. **Cammino**: sequenza di nodi $n_1, \dots, n_k, n_{k+1}$ tale che, per $i = 1, \dots, k$, n_i è il genitore di n_{i+1} . **Lunghezza del cammino**: k (numero degli archi). **Antenato e discendente**: se esiste un cammino da n a m , allora n è un antenato di m e m è un discendente di n . **Fratelli**: nodi che hanno lo stesso genitore. **Sottoalbero**: insieme costituito da un nodo n e tutti i suoi discendenti; n è la radice del sottoalbero. **Foglia**: nodo senza figli. **Nodo interno**: nodo con uno o più figli. **Profondità di un nodo**: lunghezza del cammino dalla radice al nodo. **Altezza di un nodo**: lunghezza del cammino più lungo che va dal nodo a una foglia. **Altezza dell'albero**: altezza della sua radice = profondità massima di un nodo nell'albero. **Dimensione di un albero**: numero dei nodi.

3.1 Definizione degli alberi binari per induzione strutturale

Un nodo n è un albero binario (una foglia), con radice n . Se T_0 è un albero binario con radice n_0 e n è un nuovo nodo, allora la struttura che si ottiene aggiungendo n come genitore di n_0 è un albero binario con radice n . Se T_0 e T_1 sono alberi binari con radici n_0 e n_1 , rispettivamente, e n è un nuovo nodo, allora la struttura che si ottiene aggiungendo n come genitore di n_0 e di n_1 è un albero binario con radice n . Nient'altro è un albero binario.

3.2 Definizione degli alberi n-ari per induzione strutturale

L'albero costituito da un unico nodo r senza genitori è un albero, con radice r . Se $t_1, t_2, \dots, t_n$ sono alberi, con radici, rispettivamente $r_1, r_2, \dots, r_n$, e r è un nuovo nodo, allora la struttura che si ottiene aggiungendo il nodo r come genitore di $r_1, r_2, \dots, r_n$ è un albero. Nient'altro è un albero.